

Security Assessment Report

Generated: 2026-05-06 00:05

Report ID: 21beb14f

Security Assessment Report

1. Executive Summary

An automated security review was performed against the Python project located at /workspace/uploads/3cedb7e95769 using Bandit and Sonar analysis. No codelevel vulnerabilities were identified by either Bandit or Sonar in the analyzed source code (server.py).

Additionally, the dependency scanner could not be executed because no requirements.txt file was present in the project.

Based on the available scan results, the application currently presents a Low overall security risk from a static analysis perspective. However, this assessment is limited by the absence of dependency metadata and the fact that only automated tooling results were available. As a result, the possibility of unscanned risks remains, particularly in thirdparty packages and runtime configuration.

The most significant concern is not a confirmed vulnerability in the source code, but rather the lack of dependency visibility, which prevents assurance that the software supply chain is free from known vulnerabilities.

2. Risk Overview

Findings by Severity

Critical: 0

High: 0

Medium: 0

Low: 0

Informational: 1

Risk Interpretation

Critical / High / Medium / Low: No confirmed code or dependency vulnerabilities were

identified by the provided automated scans.

Informational: The absence of a requirements.txt file prevents dependency vulnerability assessment. This limits confidence in the overall security posture because vulnerable libraries may still be present but unverified.

3. Detailed Findings

Informational Findings

Finding 1 — Missing Dependency Manifest (requirements.txt)

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A06: Vulnerable and Outdated Components

Affected Component:

Project dependency configuration for /workspace/uploads/3cedb7e95769

Description:

The scan reported that no requirements.txt file was found in the project, which prevented the dependency scanner from analyzing installed or declared Python packages.

This is not a direct software vulnerability in the application logic, but it is a security gap in the project's maintainability and assurance process.

Without a dependency manifest, it is difficult to verify whether thirdparty libraries are outdated, vulnerable, or pinned to safe versions.

Impact:

An attacker could potentially exploit unverified or outdated dependencies if they are present in the runtime environment.

From a business perspective, this reduces confidence in the security posture, complicates incident response, and increases the likelihood of supplychain risk going undetected.

It also weakens reproducibility and patch management practices.

Proof of Concept (if possible):

Not applicable as this is a configuration and assessment limitation rather than an exploitable code flaw.

Recommended Remediation:

Add and maintain a requirements.txt file or equivalent dependency manifest.

Pin all direct dependencies to knowngood versions.

Integrate dependency scanning into CI/CD using tools such as pipaudit, Safety, or SCA platforms.

Regularly update packages and review changelogs for security fixes.

If dependencies are managed differently (e.g., Poetry, Pipenv), ensure the appropriate lock file is committed and scanned.

4. Recommendations (Global)

Maintain a complete and versioncontrolled dependency manifest for all Python packages.

Integrate automated Software Composition Analysis (SCA) into the build pipeline.

Enforce dependency pinning and periodic patch reviews.

Continue using static analysis tools such as Bandit and Sonar as part of routine secure development practices.

Apply secure coding standards, including:

strict input validation and output encoding,

leastprivilege execution,

secure error handling and logging,

avoidance of unsafe deserialization and command execution patterns,

regular code review for authentication, authorization, and data handling logic.

If the application exposes network services, complement static analysis with dynamic testing and configuration review.

Ensure secrets are not embedded in code or configuration files.

Establish a vulnerability management process for thirdparty libraries and runtime environments.

5. Conclusion

The automated assessment did not identify any confirmed sourcecode vulnerabilities in the reviewed project. From the available evidence, the application appears to have a Low security risk posture.

That said, the absence of a dependency manifest is an important limitation and should be

addressed promptly. While no critical findings were discovered, dependency visibility is essential for identifying vulnerable thirdparty components and maintaining longterm security assurance. Further assessment is recommended once a complete dependency inventory is available.